

# PROPOSED SYSTEM SOLUTION & TECHNICAL SPEC

## Design Details of the Presentation and Core Intelligence Layer

### 1. Technical Stack Selection

- **Streamlit**: Selected for the UI because it allows building clean, interactive dashboard components and managing session state natively in Python.
- **SQLAlchemy & SQLite**: Provides a local relational database storage schema, mapping candidate transcripts, audio features, and reports cleanly.
- **OpenAI Whisper (Local)**: Selected for transcription as it runs locally and is robust to background noise and minor stuttering.
- **SentenceTransformer ('all-MiniLM-L6-v2')**: Used to calculate semantic embeddings; it is extremely lightweight, fast, and maintains high semantic correlation.
- **Librosa & Soundfile**: Used to load audio, calculate RMS energy, and isolate silent intervals (using energy thresholds).

### 2. Core Processing Sequence

1. User uploads WAV/MP3 file and selects reference concept.
2. Audio file is loaded via Librosa, resampled to 16,000 Hz, and converted to a mono array.
3. Whisper transcribes the mono array; total word count is calculated.
4. S-BERT computes cosine similarity between the transcript and the concept text.
5. Text is scanned for filler words to calculate the filler ratio.
6. Librosa isolates non-speech silence frames using decibel thresholds to compute the pause ratio.
7. The scoring engine deducts penalties for fillers, excessive silence, and low volume, calculating the final grade.
8. Evaluation data is logged in the database, and ReportLab compiles a PDF report.